



AI 자율 트레이딩 에이전트

Polymarket 완전 제작 가이드북

@Argona0x의 \$50 → \$2,980 성공 사례 기반

⚠ **면책 조항:** 이 가이드는 교육 목적으로만 제공됩니다. 트레이딩에는 항상 손실 위험이 있으며, 결과를 보장하지 않습니다. 각국 규정을 확인하세요.

1. 개요 및 작동 원리

@Argona0x가 공개한 시스템은 Polymarket에서 자율적으로 예측 시장을 거래하는 AI 에이전트입니다. 핵심 아이디어는 '에이전트가 살아남기 위해 수익을 내야 한다'는 게임 이론적 구조에 있습니다.

작동 사이클 (10분마다)

- 1. **Step 1:** 500~1,000 개 마켓 스캔
- 2. **Step 2:** Claude API 로 각 마켓 공정 가치(Fair Value) 추정
- 3. **Step 3:** 8% 이상 미스프라이싱 마켓 탐지
- 4. **Step 4:** Kelly Criterion 으로 포지션 크기 계산 (최대 банкроলের 6%)
- 5. **Step 5:** 주문 실행
- 6. **Step 6:** 수익에서 Claude API 비용 자동 차감 (자기 자금 조달)

💡 잔고가 \$0이 되면 에이전트는 '죽습니다'. 이 생존 압력이 에이전트로 하여금 신중한 거래를 하도록 유도합니다.

언어	Rust (빠른 실행 속도)
AI 추론	Claude API (Anthropic)
거래소	Polymarket (CLOB 기반 예측 시장)
서버	\$4.5/월 VPS (저비용 운용)
초기 자본	\$50 (테스트 기준)
48시간 결과	\$50 → \$2,980 (약 59배)

2. 사전 준비 사항

2.1 필요 계정 및 API 키

- Anthropic API 키 — claude.ai 계정 후 console.anthropic.com 에서 발급
- Polymarket 계정 — polymarket.com 가입 (Polygon 기반 지갑 필요)
- Polygon 지갑 — MetaMask 또는 전용 지갑 (USDC 보유 필요)
- VPS 서버 — Hetzner(\$4~6/월), DigitalOcean, Vultr 등

2.2 기술 스택 선택

원작자는 Rust를 사용했지만, Python이 더 쉽게 시작할 수 있습니다. Polymarket 공식 Python SDK가 존재하기 때문입니다.

언어	장점	단점						
Python	쉬운 시작, 공식 SDK 지원, 풍부한 AI 라이브러리	Rust보다 느림	Rust	빠른 속도, 낮은 레이턴시	학습 곡선 높음	TypeScript	Polymarket 공식 TS SDK 지원	Python보다 복잡

💡 초보자라면 Python을 권장합니다. Polymarket 공식 GitHub에서 Python 에이전트 프레임워크를 제공합니다: github.com/Polymarket/agents

3. 환경 설정

3.1 Python 환경 구성

```
# Python 3.9 가상환경 생성
python3.9 -m venv .venv
source .venv/bin/activate # macOS/Linux
.venv\Scripts\activate   # Windows
```

```
# Polymarket 공식 에이전트 레포 클론
git clone https://github.com/Polymarket/agents.git
cd agents
pip install -r requirements.txt
```

3.2 환경 변수 설정 (.env)

```
cp .env.example .env
```

아래 항목들을 .env 파일에 입력합니다:

```
POLYGON_WALLET_PRIVATE_KEY="0x여기에_지갑_개인키_입력"
OPENAI_API_KEY="sk-..." # OpenAI 사용 시
ANTHROPIC_API_KEY="sk-ant-..." # Claude 사용 시 (권장)
```

  개인 키를 코드에 직접 입력하거나 GitHub에 올리지 마세요! 항상 .env 파일로 관리하고 .gitignore에 추가하세요.

3.3 Polymarket 지갑 설정

7. polymarket.com 접속 후 회원가입
8. MetaMask 지갑 연결 (Polygon 네트워크)
9. USDC 를 Polygon 으로 브릿지 (bridge.polymarket.com)
10. 에이전트 전용 지갑 별도 생성 권장 (리스크 격리)

4. 에이전트 아키텍처

4.1 핵심 컴포넌트 구조

컴포넌트	역할
Market Scanner	Polymarket Gamma API로 활성 마켓 500~1000개 주기적 수집
Claude Reasoner	각 마켓 질문을 LLM에 전달 → 확률(공정가치) 추정
Mispricing Detector	공정가치 vs 현재 마켓 가격 비교 → 8% 이상 차이 탐지
Kelly Calculator	Kelly Criterion 공식으로 최적 베팅 크기 산출 (최대 6%)
Order Executor	py-clob-client로 CLOB에 주문 제출 및 체결
Cost Manager	수익에서 API 비용 차감, 잔고 모니터링

4.2 데이터 소스별 전략

Argona0x가 언급한 세 가지 마켓 카테고리별 데이터 수집 전략:

날씨 마켓 (Weather)

- NOAA API (api.weather.gov) — 무료, Polymarket 가격 업데이트 전 데이터 수집
- 예: 'Will it rain in NYC on May 10?' → NOAA 강수 예보 확률 직접 사용

스포츠 마켓 (Sports)

- ESPN API, SportsRadar, Rotowire — 부상 리포트 스크래핑
- 부상 정보가 시장에 반영되기 전 포지션 선취

₩ 크립토 마켓 (Crypto)

- 온체인 메트릭스: Dune Analytics, Nansen, Glassnode API
- 소셜センチ먼트: Twitter/X API, LunarCrush

5. 핵심 로직 구현

5.1 마켓 스캔 (Python + Polymarket API)

```
from py_clob_client.client import ClobClient
import requests
```

```
GAMMA_API = 'https://gamma-api.polymarket.com'
CLOB_HOST = 'https://clob.polymarket.com'
```

```
def scan_markets(limit=500):
    resp = requests.get(f'{GAMMA_API}/markets',
        params={'active': True, 'limit': limit, 'order': 'volume'})
    return resp.json()
```

5.2 Claude API 로 공정 가치 추정

```
import anthropic
```

```
client = anthropic.Anthropic()
```

```
def estimate_fair_value(question: str, context: str) -> float:
    msg = client.messages.create(
        model='claude-opus-4-6', # 최신 모델 권장
        max_tokens=256,
        messages=[{
            'role': 'user',
            'content': f'''
예측 시장 질문: {question}
관련 정보: {context}
이 질문의 YES 확률을 0~100 사이 숫자로만 답하세요.'''
        }]
    )
    return float(msg.content[0].text.strip()) / 100
```

5.3 Kelly Criterion 포지션 사이징

Kelly Criterion은 장기적으로 자본을 최대화하는 최적 베팅 비율을 계산합니다.

```

def kelly_fraction(p: float, market_price: float,
                  max_fraction: float = 0.06) -> float:
    '''
    p: AI 추정 확률 (예: 0.65)
    market_price: 현재 마켓 가격 (예: 0.50 = 50센트)
    '''
    if market_price <= 0 or market_price >= 1:
        return 0.0

    # Kelly 공식:  $f = (p \cdot b - (1-p)) / b$ 
    # b = 승리 시 배당률 (YES 기준)
    b = (1 - market_price) / market_price
    kelly = (p * b - (1 - p)) / b

    # 최대 벙크롤 6% 제한
    return max(0, min(kelly, max_fraction))

```

5.4 미스프라이싱 탐지 및 주문 실행

```

def find_and_trade(markets, bankroll: float):
    for market in markets:

        price = float(market['outcomePrices'][0]) # YES 가격

        context = get_context(market) # 데이터 수집
        fair_value = estimate_fair_value(market['question'], context)

        # 8% 이상 미스프라이싱 체크
        edge = fair_value - price
        if abs(edge) < 0.08:
            continue

        # 포지션 크기 계산
        fraction = kelly_fraction(fair_value, price)
        bet_size = bankroll * fraction

        # 주문 실행
        side = 'YES' if edge > 0 else 'NO'
        place_order(market['conditionId'], side, bet_size)

```

6. 자동화 스케줄러 구성

6.1 10 분 주기 실행 (Python)

```
import schedule, time
```

```
def main_loop():  
    bankroll = get_current_balance()  
    if bankroll <= 0:  
        print('Balance $0 - Agent dies. Game over.')  
        return  
  
    markets = scan_markets(limit=500)  
    find_and_trade(markets, bankroll)  
    pay_api_costs_from_profits() # 자기 자금 조달
```

```
schedule.every(10).minutes.do(main_loop)
```

```
while True:  
    schedule.run_pending()  
    time.sleep(1)
```

6.2 VPS 배포 (systemd 서비스)

서버 재부팅 시 자동 시작되도록 systemd 서비스로 등록합니다.

```
# /etc/systemd/system/polymarket-agent.service  
[Unit]  
Description=Polymarket AI Trading Agent  
After=network.target
```

```
[Service]  
User=ubuntu  
WorkingDirectory=/home/ubuntu/agents  
ExecStart=/home/ubuntu/agents/.venv/bin/python main.py  
Restart=always  
RestartSec=10
```

```
[Install]  
WantedBy=multi-user.target
```

```
# 등록 및 시작
```

```
sudo systemctl enable polymarket-agent
```

```
sudo systemctl start polymarket-agent
```

7. 리스크 관리

7.1 기본 안전장치

- 최대 단일 포지션:뱅크롤의 6% 이하 (Kelly Criterion 상한)
- 최소 Edge 기준: 8% 이상 미스프라이싱만 거래
- 잔고 \$0 시 자동 종료 (에이전트 사망 메커니즘)
- 전용 지갑 사용 (메인 지갑과 분리)

7.2 추가 권장 안전장치

```
MAX_DAILY_LOSS = bankroll * 0.20 # 일일 최대 손실 20%
MAX_POSITIONS = 10 # 동시 포지션 최대 10개
MIN_LIQUIDITY = 1000 # 최소 유동성 $1,000 마켓만
```

✓ 소액으로 시작하세요. \$20~50으로 논리 검증 후 규모를 키우는 것이 안전합니다.

7.3 비용 구조 이해

Claude API	약 \$0.003/call (claude-haiku 사용 시 비용 절감 가능)
VPS	\$4.5~6/월 (Hetzner CX11 기준)
Polymarket 수수료	Maker 0% / Taker 0% (CLOB 구조)
USDC 브릿지	Polygon 가스비 (매우 저렴, \$0.01 미만)

8. 고급 최적화

8.1 멀티 데이터 소스 통합

```
def get_context(market) -> str:
    category = market.get('category', '')

    if 'weather' in category.lower():
        return get_noaa_forecast(market)
    elif 'sports' in category.lower():
        return get_injury_report(market)
    elif 'crypto' in category.lower():
        return get_onchain_metrics(market)
    else:
        return search_news(market['question'])
```

8.2 프롬프트 엔지니어링

Claude에게 전달하는 프롬프트 품질이 수익률에 직결됩니다. 좋은 프롬프트 예시:

```
prompt = f'''
당신은 예측 시장 전문 분석가입니다.

질문: {question}
현재 마켓 가격: {current_price * 100:.1f}% (YES)
관련 데이터: {context}

다음은 고려하여 YES 확률을 추정하세요:
1. 역사적 기저율 (base rate)
2. 현재 이용 가능한 증거
3. 시장 가격과의 차이

숫자만 (0~100): '''
```

8.3 성능 모니터링

- 수익/손실 실시간 로깅 (파일 또는 DB)

- API 비용 추적 — 수익의 몇 %가 비용인지 모니터링
- 마켓 카테고리별 성과 분석 (어떤 분야가 Edge 가 높은지)
- Brier Score 로 AI 예측 정확도 측정

9. 빠른 시작 체크리스트

 이 순서대로 따라가면 48시간 이내에 첫 번째 자율 거래를 실행할 수 있습니다.

Day 1 — 환경 설정

- ☐ Anthropic API 키 발급 (console.anthropic.com)
- ☐ Polymarket 계정 생성 및 지갑 연결
- ☐ VPS 서버 구매 및 SSH 접속
- ☐ Python 환경 구성 및 Polymarket agents 클론
- ☐ .env 파일 설정

Day 2 — 개발 및 테스트

- ☐ 마켓 스캔 로직 구현 및 테스트
- ☐ Claude API 연동 및 확률 추정 테스트
- ☐ Kelly Criterion 계산기 구현
- ☐ 소액(\$5~10)으로 실제 주문 1 회 테스트
- ☐ systemd 서비스 등록

Day 3+ — 운영 및 최적화

- ☐ 10 분 주기 자동 실행 확인
- ☐ 로그 모니터링 (수익/비용)
- ☐ 데이터 소스 추가 (NOAA, ESPN 등)
- ☐ 프롬프트 최적화로 예측 정확도 개선

10. 참고 자료 및 링크

리소스	URL / 설명
-----	----------

Polymarket 공식 에이전트	github.com/Polymarket/agents
py-clob-client SDK	github.com/Polymarket/py-clob-client
Anthropic API 문서	docs.anthropic.com
Polymarket API 문서	docs.polymarket.com/api-reference
Kelly Criterion 설명	en.wikipedia.org/wiki/Kelly_criterion
NOAA 날씨 API	api.weather.gov (무료)
Polymarket 마켓 탐색	polymarket.com/predictions

최종 면책 조항

이 문서는 교육 및 정보 제공 목적으로만 작성되었습니다.

Polymarket은 국가에 따라 접근이 제한될 수 있으며(특히 미국 거주자), 거래에는 원금 손실 위험이 있습니다.

@Argona0x의 결과는 특정 시장 조건에서의 성과이며, 동일한 결과를 보장하지 않습니다.